Chapter 6: Markov Decision Processes and Dynamic Programming

Part III — Statistical Foundations of Deep Learning & Reinforcement Learning

Omid Safarzadeh

Abstract

Reinforcement learning begins not with an algorithm but with a fixed point. A Markov decision process turns "act well over time" into a precise object — the value function, the expected discounted reward from each state — and the value function is characterised by the Bellman equation, which says a state's value is its immediate reward plus the discounted value of where it leads. The reason this equation can be *solved* is one clean fact: the Bellman operator is a contraction, shrinking distances by the discount factor γ, so by Banach's theorem it has a unique fixed point that repeated application converges to. This chapter proves the contraction, reads value and policy iteration off it, and — when the transition model is unknown — estimates the value from sampled experience by Monte Carlo and temporal-difference learning, the latter being the stochastic approximation of Chapter 3 in disguise.

Contents

1 Markov decision processes and the value function 1

2 The Bellman operator is a contraction 2

3 When the model is unknown: Monte Carlo and TD 2

1 Markov decision processes and the value function

Definition 1.1 (MDP)**.** A **Markov decision process** is $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$: states $\mathcal{S}$, actions $\mathcal{A}$, a transition kernel $P(s' \mid s, a)$, a reward $r(s, a)$, and a **discount** $\gamma \in [0, 1)$. A **policy** $\pi(a \mid s)$ chooses actions; its **value function** is the expected discounted return

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^{t} r(s_t, a_t) \,\middle|\, s_0 = s\right].$$

The sum telescopes into a one-step relation, because the return from s is the reward now plus γ times the return from the next state.

Definition 1.2 (Bellman equations)**.** The value of a fixed policy satisfies the **Bellman expectation equation**

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s)\Big[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^{\pi}(s')\Big],$$

and the optimal value the **Bellman optimality equation**

$$V^*(s) = \max_a \Big[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s') \Big].$$

The left panel of the figure shows an optimal value function V^* on a slippery gridworld, together with the greedy policy it induces — each arrow points to the action that maximises the bracket above.

2 The Bellman operator is a contraction

Write the optimality equation as a map on value functions: the **Bellman optimality operator** T sends a value function V to

$$(TV)(s) = \max_a \Big[r(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s') \Big].$$

V^* is exactly a fixed point of T. The following is why that fixed point exists, is unique, and can be found by iterating.

Theorem 2.1 (T is a γ-contraction in the sup norm)**.** For any value functions V, U, $\|TV - TU\|_\infty \le \gamma \, \|V - U\|_\infty$.

Proof. Fix a state s. Using $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$,

$$|(TV)(s) - (TU)(s)| \le \max_a \gamma \sum_{s'} P(s' \mid s,a) \, |V(s') - U(s')| \le \gamma \, \|V - U\|_\infty \max_a \sum_{s'} P(s' \mid s,a).$$

The rewards cancel, and $\sum_{s'} P(s' \mid s,a) = 1$, so the right side is $\gamma \|V - U\|_\infty$. Taking the max over s gives the claim. □

Theorem 2.2 (Value iteration converges)**.** T has a unique fixed point V^*, and $V_{k+1} = TV_k$ satisfies $\|V_k - V^*\|_\infty \le \gamma^k \|V_0 - V^*\|_\infty \to 0$.

Proof. A contraction on the complete metric space $(\mathbb{R}^{|\mathcal{S}|}, \|\cdot\|_\infty)$ has a unique fixed point and its iterates converge to it (Banach). Applying the contraction k times to V_0 and $V^* = TV^*$ gives $\|V_k - V^*\|_\infty = \|T^k V_0 - T^k V^*\|_\infty \le \gamma^k \|V_0 - V^*\|_\infty$. □

The middle panel is this bound in action: the sup-norm error falls geometrically, staying under the γ^k line. **Policy iteration** is the other route to V^* — alternate evaluating the current policy (solving the linear Bellman expectation equation) and acting greedily with respect to that value — and it converges in finitely many steps, because there are finitely many policies and each step strictly improves.

3 When the model is unknown: Monte Carlo and TD

Dynamic programming needs P and r. Reinforcement learning usually does not have them, only sampled experience, so it *estimates* the value. Two ways, and they are the two integral-estimation styles of Chapter 4 applied to the return. **Monte Carlo** evaluation waits until an episode ends and moves each visited state's estimate toward the observed return G:

$$V(s) \leftarrow V(s) + \alpha \big(G - V(s) \big).$$

Temporal-difference learning does not wait; it bootstraps, moving toward the one-step estimate $r + \gamma V(s')$ immediately:

$$V(s) \leftarrow V(s) + \alpha\big(\underbrace{r + \gamma V(s') - V(s)}_{\text{TD error}} \big).$$

TD(0) is Robbins–Monro stochastic approximation (Chapter 3) on the Bellman expectation equation: the TD error is a noisy, unbiased sample of "how wrong the current value is," and the step-size conditions $\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$ are exactly what guarantee convergence. Because TD updates every step and uses its own value estimate as a target, it typically has lower variance than Monte Carlo — the right panel, where TD(0) reaches a given accuracy in far fewer episodes than constant-α Monte Carlo on the same random walk.

Summary

An MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ defines the value function V^π, characterised by the Bellman equations; the optimal value V^* is the fixed point of the Bellman optimality operator T. T is a γ-contraction in the sup norm, so by Banach it has a unique fixed point and value iteration converges geometrically, $\|V_k - V^*\|_\infty \le \gamma^k \|V_0 - V^*\|_\infty$; policy iteration converges in finitely many strict-improvement steps. Without a model, the value is estimated from experience: Monte Carlo moves toward the realised return, TD(0) bootstraps toward $r + \gamma V(s')$ — the latter is the Chapter 3 stochastic approximation of the Bellman equation, lower-variance than Monte Carlo, and the foundation for the policy-gradient methods of Chapter 7.